

PROBLEM SET / 08

Measure the task, not the reputation

A projected benchmark suite for use-case-specific AI capabilities

Boomer Rawlings | Independent individual work

Proposed after the original submission and grading dates. Status: Not started.

Abstract

A model benchmark supports a deployment decision only when its tasks, allowed tools and scoring rules resemble the intended use. This proposal develops a modular assessment suite for scientific assistance, software maintenance, defensive code analysis, multilingual understanding and appropriate refusal. The projected system will define each capability as an observable task contract with independent evidence of success. It will publish capability profiles and cost distributions rather than collapse every dimension into a single ranking. [1]

The primary research question is whether domain-specific tasks reveal capability differences and failure modes that broad public scores obscure. A second question asks how benchmark validity changes when an agent helps generate its own tasks. The proposed design separates task authorship, solution generation and adjudication; retains hidden family-level holdouts; and tests whether conclusions remain stable under alternate rubrics, tool budgets and model revisions. No task generation or evaluation has yet occurred.

Status and scope

NOT STARTED. This paper is a projected solution, study design and implementation roadmap. No benchmark suite, model run, scored dataset or capability result is claimed. Sample counts and decision margins are proposed planning values.

This is planned independent individual work by Boomer Rawlings after the original submission and grading dates. The government-relevant framing motivates the task categories; all proposed examples use public, licensed or authored material. The project does not claim operational certification or access to government data.

1. Construct validity and related work

A useful assessment starts with the decision it must inform. Can an assistant find a statistical error without inventing missing data? Can a coding agent repair a compiler-sensitive numerical routine while preserving behavior? Can a language model retain implication and uncertainty across languages? These are different constructs. They require different reference evidence and cannot be represented faithfully by one generic accuracy score.

HELM motivates evaluation across multiple scenarios and metrics under standardized conditions. This proposal adopts that emphasis on transparent profiles, but narrows each task to a concrete workflow and decision. SWE-bench motivates repository-level issue resolution with executable checks; its original Python-focused design is a methodological reference, not evidence that the same tasks measure competence in legacy languages. [2] [3]

Datasheets for Datasets motivates documenting collection, composition and intended use. Here that documentation will include construct definitions, task-family provenance, author access to model outputs and the conditions under which a score should not guide deployment. LLM-as-a-judge research motivates blinded comparisons and bias checks, while the proposed suite will prefer executable or independently adjudicated evidence wherever possible. [4] [5]

Research questions

RQ1: Are model capability profiles stable across task families within a use case? RQ2: Which tool and cost constraints change the model ordering? RQ3: Do domain-specific failures add information beyond broad public benchmark scores? RQ4: Does self-generated evaluation overestimate capability compared with independently authored holdouts? These questions require task-level results and uncertainty, not anecdotal demonstrations.

The initial implementation will focus on two vertical slices, scientific reasoning and software maintenance, before adding the remaining tracks. This is a feasibility choice: validity in a small, well-annotated suite is more useful than superficial coverage. The full roadmap retains all task families from the brief, with separate admission criteria and reviewers.

2. Proposed task taxonomy and contracts

Track	Projected task	Independent success evidence
Scientific assistant	Analyze a supplied dataset and defend a conclusion	Executable analysis; hidden data checks; expert rubric
Software maintenance	Repair an authored Fortran/Ada or Python defect	Compiler matrix; hidden regression tests; review
Defensive analysis	Inspect an owned toy binary or source package	Known seeded weakness; bounded artifact rubric
Language understanding	Interpret culturally grounded, contextual passages	Bilingual rationale and adjudicated semantic labels
Appropriate refusal	Respond to matched permitted/restricted task pairs	Policy-grounded labels; helpfulness and boundary scores

Each task contract will declare the user goal, input artifacts, tool permissions, time and token budget, output format, scoring components and disqualifying behavior. An immutable task ID will identify the exact contract version. The harness will retain the final artifact, relevant tool actions, errors and resource accounting. Private reasoning text will not be required as a scoring input; observable decisions and artifacts are sufficient.

A scientific task may contain a planted confound, unit mismatch or unsupported causal conclusion. The score will distinguish correct computation from warranted interpretation. A coding task will require a minimal repair that passes hidden tests and preserves an interface, rather than reward matching a reference patch. Defensive analysis will remain inside isolated, owned examples; it will not involve probing real targets or publishing operational exploit instructions.

Language tasks will include ambiguity, implicature, register and context, with annotators documenting acceptable interpretations. Refusal tasks will include benign analysis of sensitive-looking text as well as requests that require boundaries. The goal is to measure both over-refusal and unsafe compliance using authorized, non-operational fixtures, not to optimize methods for bypassing safeguards.

3. Dataset construction and contamination control

The proposed authoring process begins with a construct map and task-family templates reviewed by domain specialists. Authors will create a task, reference evidence and rubric before inspecting evaluated model outputs. Every artifact will receive a rights record and a provenance entry describing whether it was human-authored, model-assisted, adapted from a public source or generated by a deterministic script.

Separate design, development and test

A pilot target of 20 tasks in each of the first two tracks will calibrate instructions and scoring. A later target of 100 tasks per admitted track will be revised using pilot variance, task-family diversity and reviewer cost. Families, repositories and source documents will be assigned wholly to one partition. Superficial paraphrases of the same problem will not be treated as independent examples.

Public tasks will support reproducibility; a separately governed rotating holdout will test generalization to unseen task families. Publication dates, source commits and known benchmark overlap will be recorded. Similarity scanning can identify obvious duplicates but cannot prove absence from unknown training corpora. Results will therefore distinguish contamination controls from claims of contamination-free evaluation.

Self-evaluation experiment

To study recursive self-evaluation, one arm will let an agent propose task instances from a declared domain. Those tasks must pass independent validity review before scoring. The generating model, sibling model families and independent models will then be evaluated on both generated tasks and independently authored holdouts. The main comparison is the change in the generator advantage across those two sets.

Generated tasks that are invalid, ambiguous or unsolvable will be counted and characterized, not quietly removed from the generation yield. Scoring the same model against its own unreviewed answers would be circular; independent adjudication and holdouts are essential. A high self-generated score can indicate alignment with the generator distribution rather than broad competence.

4. Harness and scoring methodology

The planned harness will execute versioned tasks in isolated workspaces with bounded tools and deterministic artifact collection. A model adapter will normalize requests and return output plus usage metadata. The evaluator will operate in a separate process with hidden checks unavailable to the agent. Network access, package installation and filesystem scope will be specified per task and logged when allowed.

Scoring layer	What it measures	Proposed protection
Executable checks	Functional correctness and invariants	Hidden tests; independent fixtures; tamper checks
Artifact rubric	Completeness, support and interpretation	Blinded dual review; adjudication
Behavior trace	Authorized actions, recovery and uncertainty	Observable action labels; no reasoning-text demands
Resource record	Time, tokens, tool calls and memory	Count retries, failures and setup separately
Judge assistance	Suggested rubric labels at scale	Human calibration; bias and transfer checks

For coding tasks, a pass requires both issue-specific checks and regression preservation. A test harness that the agent can modify is not trusted as the sole oracle. Scientific tasks will preserve input data hashes and validate calculations independently. Narrative scoring will distinguish factual correctness, attribution and uncertainty; stylistic fluency will not compensate for incorrect claims.

A partially completed artifact may receive component scores, but task success will use prespecified criteria. The suite will report pass@1 under one fixed budget as its primary operational measure. Additional attempts will be labeled separately with total cost; best-of-many results will not be described as single-attempt reliability.

Automatic judges will receive blinded outputs and rubrics with order randomization. A held-out audit will compare their labels with independent reviewers. If judge agreement is inadequate in a task family, that family will use human scores or remain outside automatic aggregation. Judge confidence will never replace evidence that the rubric was applied correctly.

5. Experimental analysis and decision utility

Models will be compared under the same task budgets, tool permissions and artifact constraints. The initial proposed panel includes at least three distinct model families and a deterministic or specialist baseline where meaningful. Versions and access dates will be pinned. A provider update creates a new condition; it does not silently replace an earlier measurement.

Primary results will be per-track success, critical error rate, justified abstention and cost per successful task. Cost per success will include failed attempts in its numerator. Latency distributions and tail failures will accompany medians. A deployment-oriented view may apply stakeholder-defined minimum quality and maximum cost gates, but the underlying raw profile will remain available.

Uncertainty and comparisons

The task family is the principal resampling unit. Cluster bootstrap intervals will preserve related variants and repeated seeds. Pairwise differences will be computed on shared tasks, with a prespecified primary comparison and correction or explicit exploratory labeling for additional comparisons. A leaderboard difference smaller than its uncertainty will be reported as unresolved.

The proposed analysis will test sensitivity to rubric weights, exclusion rules and tool budgets. Rankings that reverse under reasonable weight changes should guide a task-specific selection discussion rather than support a universal best-model claim. For scientific tasks, expert ratings will be analyzed alongside executable correctness; disagreement between them may reveal invalid shortcuts or overly restrictive references.

Decision rules

Before final runs, each use case will define unacceptable failure classes and a minimum evidence threshold. No single sample count is sufficient for rare critical failures: observing zero events in a small suite does not establish negligible risk. The report will state the finite-sample uncertainty and the population represented by each task family. Claims of government readiness or general intelligence are outside this study.

6. Roadmap and validity safeguards

Phase	Planned work	Exit evidence
A: construct map	Select decisions, task families and admissible tools	Expert-reviewed construct and scoring map
B: vertical pilot	Author scientific and maintenance tasks	Solvability checks; dual-review agreement
C: harness	Implement isolation, model adapters and hidden scoring	Replayable tasks; tamper and failure tests
D: expanded suite	Admit defensive, language and refusal tracks	Qualified reviewers; rights/provenance records
E: frozen study	Run comparisons and self-evaluation experiment	Complete cost/trace records; held-out analysis
F: release cycle	Publish public suite and maintain rotating holdout	Versioned release and change-impact report

Tasks will be retired or revised when they leak, become ambiguous after tool changes or stop representing the target workflow. A revision creates a new task version and a bridge evaluation on unaffected tasks. Benchmark maintenance is part of the design: a changing model landscape makes an unversioned aggregate score difficult to interpret.

Reviewer expertise and disagreement are central validity risks. The protocol will document domain experience, language proficiency, adjudication and compensation where applicable. Task authors will not be the sole judges of outputs that challenge their expected solution. Independent alternate valid solutions will be admitted by a documented appeal process.

A future product page could display a task contract, anonymized candidate artifacts and an inspectable scoring breakdown. The present page provides the proposal and roadmap only. No capability dashboard or working evaluation harness exists in this project yet. The next concrete step is to select the two pilot use cases and write reviewed task contracts.

7. References and planned research contribution

Primary sources consulted October 7, 2026. These works motivate evaluation design and validity controls. They do not establish the performance of any model on this proposed suite.

[1] SCADS 2026 Problem Book, Project 8, pp. 10-11. User-supplied research brief.

[2] Liang et al. (2023). Holistic Evaluation of Language Models. TMLR.
<https://arxiv.org/abs/2211.09110>

[3] Jimenez et al. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR.
<https://arxiv.org/abs/2310.06770>

[4] Gebru et al. (2021). Datasheets for Datasets. Communications of the ACM.
<https://arxiv.org/abs/1803.09010>

[5] Zheng et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS.
<https://arxiv.org/abs/2306.05685>

Planned publication package

The intended release will contain task contracts, public inputs with rights records, reproducible environments, reference-check specifications, reviewer rubrics, anonymized adjudication labels where permitted, model-run manifests and scripts for uncertainty analysis. Hidden evaluation data will be governed separately, with documented access and a public description of its coverage.

A useful outcome may be a narrow capability boundary rather than a winning model. Examples include a system that computes accurately but overstates causality, repairs common syntax while failing numerical invariants, or translates literal content while missing contextual implication. The proposed suite is designed to make such distinctions inspectable.

Current status: Not started. The benchmark design is projected; implementation and evaluation have not begun. No empirical score, comparative ranking or operational adoption recommendation is reported.